

rdma-forensics

Host-side RDMA forensics for RoCEv2 fabrics: verbs, counters, and casefiles

binaryhustleio
generated 2025-09-13
source: docs/manual.md in the rdma-forensics repository

Contents

Preface

Part I: The mental model

1. Where verbs sit in the stack
2. The objects
3. Queue pair types and the RC state machine
4. The handshake
5. Two-sided versus one-sided
 - 5a. Many NICs in one node
6. Completions and polling
7. What the fabric does to the host, in counters

Part II: The four investigations

- Investigation 1: pingpong and the state machine
- Investigation 2: one-sided WRITE and READ
- Investigation 3: atomics
- Investigation 4: many queue pairs and fault injection

Part III: Runbooks

- R1. Soft-RoCE VM bring-up
- R2. Pod preflight (run first on every pod, before anything else)
- R3. One experiment, start to finish
- R4. Fault injection with guaranteed undo
- R5. Packet capture and decode

Part IV: Reference

- Wire format
- RC opcodes decoded by `bth_parse.py`
- `modify_qp` attribute masks, by transition
- Common errors and what to ask first
- Repository map
- Reading list

Preface

This manual is the offline companion to the `rdma-forensics` repository, the host-side half of a two repo forensic toolkit. `fabric-forensics` covers what the switches do (PFC, ECN, DCQCN, ECMP). This repository covers what the NIC and the verbs stack do with the transport those fabrics carry: how a queue pair is built, how memory is exposed, how one-sided operations land, and which host counters move when the fabric misbehaves.

Most cluster problems live in the gap between those two views. A switch counter says packets were marked; a NIC counter says a queue pair went to retry; a training job says it is 30% slow. The casefiles in this repository are the method for connecting those three statements with measurements rather than inference.

The repository is the lab; this document is the model, the runbooks, and the reference. Every command in a runbook exists in the repo with a `--dry-run` flag so it can be rehearsed before a metered pod session.

Method, in one paragraph: hypothesis before hardware. Baseline snapshot before every experiment. Verbs code written by hand; the first working version is committed and tagged `<name>v1` (for example `pingpongv1`) with its measured number, and the refactor follows as a separate commit, so the refactor history is visible through `git diff` . Nothing enters the results table until it has been measured on both Soft-RoCE and ConnectX.

Part I:

1. Where verbs sit in the stack

A RoCEv2 packet on the wire is Ethernet, IP, UDP to destination port 4791, then the InfiniBand transport headers, then payload, then a 4-byte ICRC. The fabric sees UDP flows. The host sees work queues.

```

application
  | verbs API (libibverbs, pyverbs)
  v
+-----+
| user-space provider (mlx5, | posts descriptors straight to NIC memory
| rxe)                      | via mmap'd doorbells; no syscall per op
+-----+
  |
  v
NIC (or the rdma_rxe software emulation in the kernel)
  |
  v
Ethernet / IP / UDP 4791 / BTH / [RETH] / payload / ICRC

```

Two consequences shape everything else in this toolkit:

1. The kernel is not on the data path. Once a queue pair is connected, posting a work request is a memory write and a doorbell ring. This is why latencies are single-digit microseconds on real hardware.
2. Reliability is in the NIC, not in a software stack. Retries, sequence numbers, and acknowledgements are handled by the transport engine. When the fabric drops a packet, the evidence appears in NIC hardware counters, not in netstat.

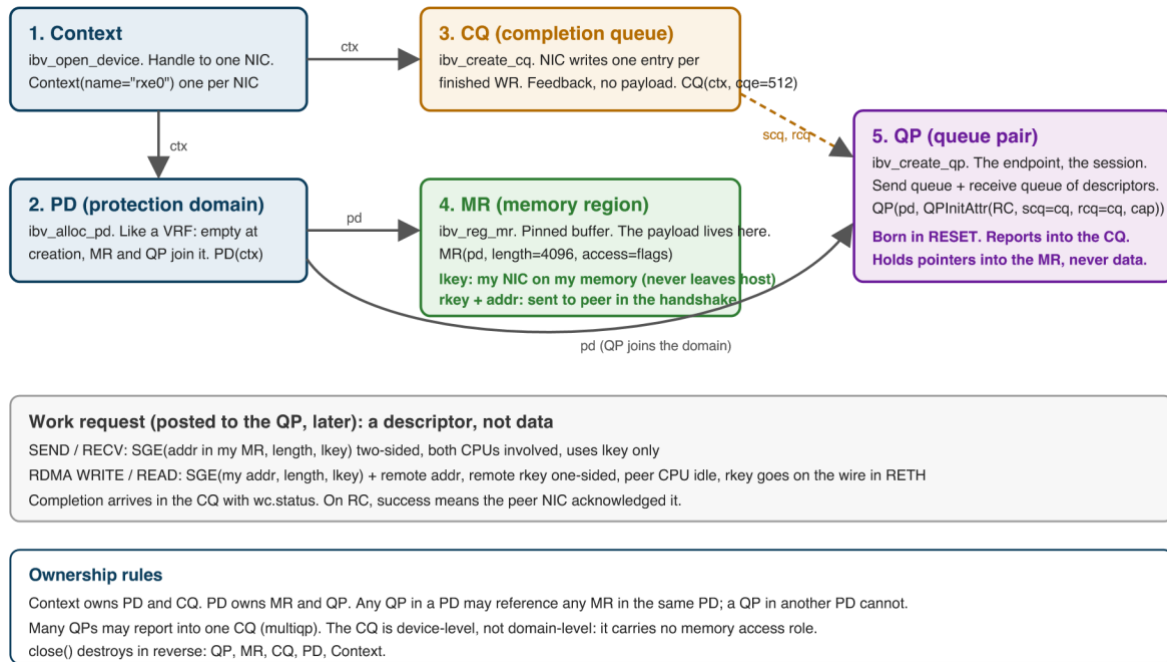
2. Objects

Every verbs program creates the same six objects in the same order.

object	verbs name	what it is	analogy
Context	ibv_context	open handle to one device	a socket family
Protection Domain	ibv_pd	isolation boundary; MRs and QPs in different PDs cannot touch	a VRF
Memory Region	ibv_mr	pinned, registered buffer with lkey and rkey	a route that has been installed
Completion Queue	ibv_cq	where the NIC reports finished work	an interface counter you poll
Queue Pair	ibv_qp	a send queue plus a receive queue; the endpoint	a BGP session
Work Request	ibv_send_wr / ibv_recv_wr	one operation posted to a queue	one update

The verbs object model: what open() builds, in order

Each box is one pyverbs class. The arrow into a box is its first constructor argument: the thing it lives inside.



The Memory Region is the object with no networking analogy that survives contact. Registering memory pins the pages so the NIC can DMA to them and produces two keys. The local key (lkey) authorizes the local NIC to touch the buffer. The remote key (rkey) is handed to the peer and authorizes the peer's NIC to read or write it without involving this host's CPU. Leaking an rkey is leaking write access to your RAM.

3. Queue pair types and the RC state machine

This toolkit uses Reliable Connected (RC) queue pairs exclusively. RC is the transport under NCCL, MPI, NVMe-oF, and GPUDirect. An RC QP talks to exactly one peer QP and guarantees in-order delivery with retransmission.

A QP is born in RESET and must be walked through three transitions before it can carry traffic. Each transition is one call to `ibv_modify_qp` with an attribute struct and a mask naming exactly which fields are valid. A wrong mask returns `EINVAL` with no further hint. This is the single most common failure for a newcomer, and it is why `verbs/connection.py` models the machine as an explicit class.

```
RESET --to_init()--> INIT --to_rtr(remote)--> RTR --to_rts()--> RTS
|
any completion error -----> ERR
```

RESET to INIT binds the QP to a port and declares what remote operations it will accept. Four fields: state, pkey index, port number, access flags.

INIT to RTR (ready to receive) tells the QP who its peer is. This is the transition that needs the handshake data: destination QP number, the PSN the peer will start sending with, the path MTU, the address vector (the peer GID, the local GID index, hop limit), the maximum number of inbound RDMA READ or atomic operations, and the minimum RNR timer. Seven fields.

RTR to RTS (ready to send) arms the sender-side retry machinery: our own starting PSN, the ACK timeout, the retry count, the RNR retry count, and the maximum outbound READ or atomic operations. Six fields.

Fabric note. The timeout field is an exponent: the timer is 4.096 microseconds times two to the power of the value. The `rc_pingpong` default of 14 is about 67 milliseconds. The retry count default of 7 means a single lost packet with no ACK costs at least seven timeouts before the QP goes to ERR. On a lossy fabric this is what turns 1% loss into a job that appears hung. Investigation 4 measures it.

4. The handshake

Before either QP can reach RTR, each side must obtain five values from the other:

field	width	why the peer needs it
qpn	24 bits	destination QP number in every BTH
psn	24 bits	the first sequence number to expect
gid	16 bytes	the L3 address of the port (for RoCEv2, an IPv4-mapped or IPv6 address)
rkey	32 bits	authorizes one-sided operations on the peer buffer
addr	64 bits	virtual address of that buffer

There is no in-band way to do this because the QPs are not connected yet. Every RDMA program exchanges these over TCP or a rendezvous service. `verbs/handshake.py` packs them into a 36-byte big-endian record and swaps it over one TCP connection. It is plumbing, and it is the one piece of the connection path that contains no verbs.

5. Two-sided versus one-sided

SEND and RECV are two-sided. The receiver must post a RECV work request before the SEND arrives. If no RECV is posted, the receiver NIC returns an RNR NAK (receiver not ready) and the sender backs off for the minimum RNR timer and retries. Both CPUs are involved, and both sides get a completion.

RDMA WRITE and RDMA READ are one-sided. The initiator names the remote address and rkey in a RETH header. The target NIC performs the DMA. The target CPU is never told. Only the initiator gets a completion. Under a WRITE, data flows one way and an ACK returns; under a READ, a request goes out and response packets carry the data back.

WRITE with immediate is the hybrid: a one-sided write that also consumes a RECV on the target and delivers a 32-bit immediate value in the completion. It is how a producer tells a consumer that a buffer is ready without a second round trip.

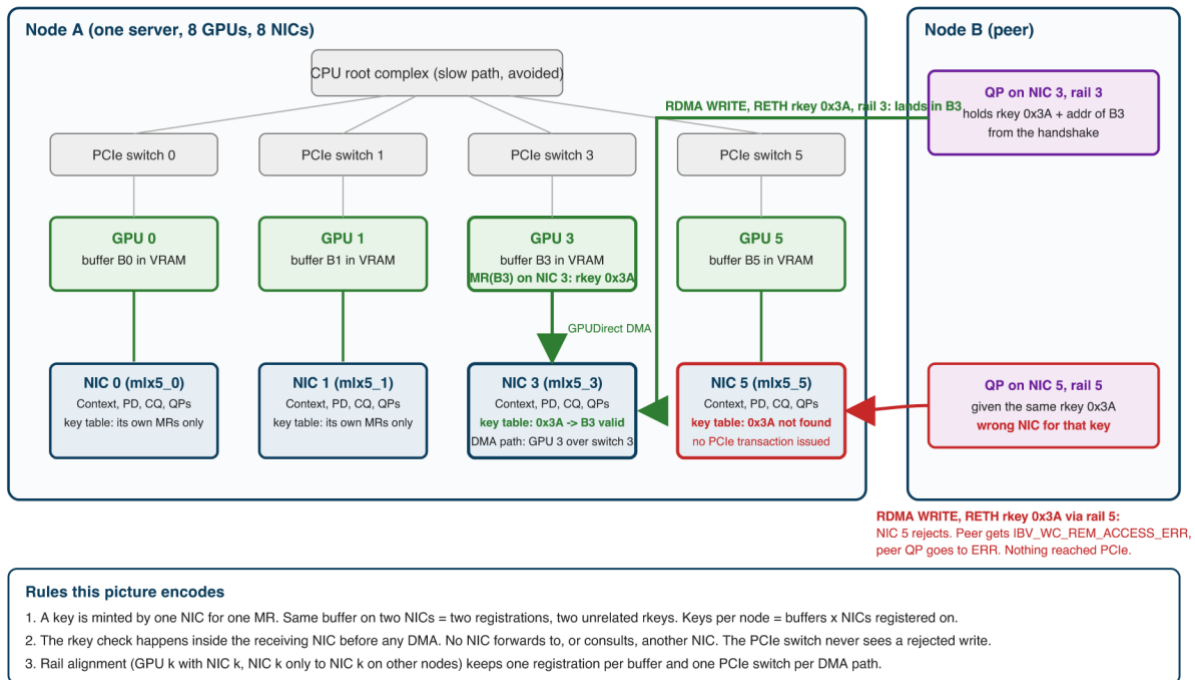
Atomics (fetch-and-add, compare-and-swap) operate on 8 aligned bytes at the target and return the previous value. The target NIC serializes, which is what makes a remote counter or lock correct across many initiators.

5a. Many NICs in one node

Every NIC is its own Context, with its own PD, CQs, MRs and QPs. A key is minted by one NIC for one MR, so the same GPU buffer registered on two NICs has two unrelated rkeys. An rkey presented to a NIC that did not mint it is rejected inside that NIC before any PCIe transaction, and the initiator sees IBV_WC_REM_ACCESS_ERR. Rail alignment, GPU *k* paired with NIC *k*, keeps one registration per buffer and one PCIe switch per DMA path.

Eight-NIC node: keys are per MR per NIC, and an rkey is only valid at the NIC that minted it

Simplified to four of eight rails. Each GPU pairs with the NIC on its PCIe switch (rail alignment). Each NIC is its own Context with its own PD, CQ, MRs, QPs.



6. Completions and polling

The NIC reports finished work by writing a completion entry to the CQ. The application either spins on the CQ (busy polling: lowest latency, one core burned) or blocks on a completion channel (an fd that becomes readable when the CQ has entries: higher latency, CPU freed).

Every benchmark number in this repository records which mode produced it. Busy polling on SoftRoCE makes the software path look faster than it is because the poll loop and the kernel emulation thread compete for the same VM cores. The question "why is this slow" is not answerable until the backend and the polling mode are both known.

A completion carries a status. Anything other than success must be checked on every completion, not just the first, because the QP has already moved to ERR by the time the status arrives and every subsequent work request will flush with a failure.

7. What the fabric does to the host, in counters

The link between this repo and fabric-forensics is `/sys/class/infiniband/<dev>/ports/<n>/`. Two directories matter.

`counters/` holds the port-level IB standard counters: packets and bytes in each direction, receive errors, and `port_xmit_wait`, which ticks when the port had data to send and no credits. On a RoCE fabric, credits are PFC pause frames, so `xmit_wait` is the host-side shadow of pause storms. `hw_counters/` holds the vendor transport counters. On `mlx5` the ones that matter:

counter	meaning	fabric cause
<code>out_of_sequence</code>	a packet arrived with an unexpected PSN	loss upstream of this port
<code>packet_seq_err</code>	this side detected a PSN gap and NAKed	loss on the inbound path
<code>local_ack_timeout_err</code>	our retry timer fired	ACKs lost, or a stalled peer
<code>rn timer_err</code>	peer had no RECV posted	application pacing, not fabric
<code>np_cnp_sent</code>	we sent a congestion notification packet	ECN marks received; DCQCN active
<code>rp_cnp_handled</code>	we received a CNP and reduced rate	our flows were marked upstream
<code>roce_adp_retrans</code>	adaptive retransmission fired	selective repeat in newer firmware

Soft-RoCE exposes a smaller set with different names. `forensics/counters.py` snapshots both directories to JSON before and after each experiment, and the diff is the evidence in every casefile.

Part II: Investigations

Each investigation has a module, a tagged first working version, a casefile, and a fabric question the casefile must answer. Reading references are the rdma-core man pages (`ibv_post_send(3)` , `ibv_modify_qp(3)` , `ibv_reg_mr(3)`), the `rc_pingpong` example source in rdma-core, and the InfiniBand Architecture Specification volume 1, chapter 9 (transport) and chapter 10 (verbs).

Investigation 1: pingpong and the state machine

Module `verbs/pingpong.py` , built on `verbs/connection.py` .

Goal Reproduce `ibv_rc_pingpong` in pyverbs and match its usec/iter within 20% on rxe. **Sequence**

1. Implement `RCConnection.open()` : context, PD, CQ, MR, QP in RESET.
2. Implement `to_init()` . Verify with `query_state()` that the HCA agrees.
3. Post RECVs before the handshake. Then `to_rtr()` , `to_rts()` .
4. Client posts SEND, polls; server polls, posts SEND back. Loop.
5. Measure with busy polling, then with a completion channel.

First pass One function, hard-coded sizes, no error checks. When it produces a number, commit and tag it `pingpong_v1` with that number in the tag message, then refactor. The diff between the tag and the refactor is part of the record.

Fabric question Compare the pyverbs number with `ibv_rc_pingpong` on the same VM. Where does the difference come from, and does it change with message size?

Failure to induce Set the no-recv flag on the server with `forensics/inject.py` . Watch `nrn_nak_retry_err` climb. Then clear it and watch the run complete. This is what a stalled consumer looks like from the sender's NIC.

Investigation 2: one-sided WRITE and READ

Module `verbs/write_read.py` .

Goal Measure WRITE and READ latency and bandwidth across sizes on rxe and on mlx5, and prove with a packet capture where the RETH header appears.

Sequence

1. Server fills its buffer with a known pattern and posts nothing.
2. Client WRITES a different pattern, polls its completion, then READs it back and verifies.
3. Repeat for WRITE_WITH_IMM and observe that the server now needs a RECV.
4. Capture with `tcpdump -i <dev> udp port 4791 -w write.pcap` , decode with `forensics/bth_parse.py` .

Expected observation A 64 KiB WRITE at path MTU 1024 is 64 packets: one RDMA_WRITE_FIRST carrying RETH, 62 MIDDLE, one LAST with ack_req set. RETH appears exactly once. A READ is one request with RETH and a stream of RESPONSE packets.

Fabric question Which of these packets does ECMP hash? All packets of one QP share a UDP source port, so a 64 KiB WRITE is one flow. How does that interact with the flowlet and per-packet spraying options on the switches in fabric-forensics?

Investigation 3: atomics

Module `verbs/atomics.py` .

Goal Prove that N concurrent fetch-and-add initiators produce exactly N times iters on the target counter, and measure the acquire latency of a compare-and-swap spinlock.

Sequence

1. Server exposes a zeroed 8-byte aligned counter.
2. One client, one process: FADD loop, check final value.
3. Two clients (two processes on the same node in loopback, or two nodes).
4. CAS lock: spin on CAS(0 to 1), write 0 back, measure.

Fabric question Atomics are round trips by construction. What is the latency floor set by the fabric diameter, and what does that say about proposals to build a lock service on RDMA across a spine?

Investigation 4: many queue pairs and fault injection

Module `verbs/multiqp.py` .

Goal Scale from 1 to N QPs on one port, measure aggregate bandwidth and completion fairness, then inject loss and watch which QPs pay for it.

Sequence

1. Refactor RConnection to share a PD and CQ across QPs (the refactor is the exercise).
2. Extend the handshake to exchange N records.
3. Round-robin WRITES, one poll loop, per-QP completion accounting.
4. `forensics/inject.py loss --pct 1 --confirm` on the client netdev, rerun, diff counters, undo.
5. Repeat with MTU lowered on one side.

Fabric question Each QP is a distinct UDP source port and therefore a distinct ECMP flow. Show it in the capture. Then argue whether N QPs per GPU pair is a load-balancing feature or a congestioncontrol liability under DCQCN.

Part III: Runbooks

Every command below can be run with `--dry-run` first.

R1. Soft-RoCE VM bring-up

```
sudo bash setup/softroce_vm.sh --dry-run
sudo bash setup/softroce_vm.sh
```

The script installs packages, loads `rdma_rxe`, adds an `rxe` link on the default-route netdev, and runs a loopback `ibv_rc_pingpong`. It ends with a PASS/FAIL summary. Rerunning is safe.

If the port is not ACTIVE, the netdev has no carrier or no IP. Fix that first.

R2. Pod preflight (run first on every pod, before anything else)

```
bash setup/pod_preflight.sh --dry-run bash setup/pod_preflight.sh
# single node, loopback perftest bash setup/pod_preflight.sh --peer 10.0.0.2
# two nodes, prints the commands
```

Output: device inventory, every GID with its type (choose the RoCE v2 entry for `-g`), link rate, active MTU, and a JSON baseline snapshot in `forensics/baseline/`. Copy the gid index and device name into the casefile Environment table immediately.

R3.

```
make casefile NAME=write_mlx5_loopback python -m
forensics.counters snapshot --out before.json
# run the experiment
python -m forensics.counters snapshot --out after.json python -m
forensics.counters diff before.json after.json --nonzero --markdown python -m
bench.plots results.csv --op write --out casefiles/write_mlx5_loopback/ plot.svg
```

Paste the diff table and the plot into the casefile. Write the finding. Then the fabric paragraph. Commit with a measurement verb.

R4. Fault injection

```
python -m forensics.inject --dry-run loss --dev eth0 --pct 1
python -m forensics.inject loss --dev eth0 --pct 1 --confirm
# run python -m forensics.inject loss --dev eth0 --undo --
confirm
```

The MTU helper saves the previous value so undo works from a fresh shell. `netem` applies to egress only; to see loss on the receive path, apply it on the peer.

R5. Packet capture/ decode

```
sudo tcpdump -i eth0 -s 0 udp port 4791 -w capture.pcap python -m  
forensics.bth_parse capture.pcap | head -50 python -m  
forensics.bth_parse capture.pcap --json > capture.jsonl
```

On mlx5 with kernel bypass, tcpdump on the netdev sees nothing. Use `tcpdump -i <rdma_dev>` if the driver exposes an RDMA capture interface, or capture on the switch mirror port. On rxe every packet between two hosts is visible on the netdev.

rxr loopback cannot be captured: when both QPs live on the same rxr device the driver delivers packets internally and they never reach the netdev, so a tcpdump of a loopback `ibv_rc_pingpong` is empty. Every capture in this repository therefore comes from two VMs (`rdma1` and `rdma2`) or from the synthetic fixture in `tests/fixtures`. Wireshark decodes UDP 4791 as RoCEv2 natively with the InfiniBand dissector; filter with `infiniband`.

Part IV: Reference

Wire format

```
Ethernet 14 | IPv4 20 | UDP 8 (dport 4791) | BTH 12 | [RETH 16] | [AETH 4] |
payload | ICRC 4
```

BTH bit layout:

bytes	field	notes
0	opcode	high 3 bits transport (000 RC), low 5 operation
1	SE, M, Pad(2), TVer(4)	solicited event, migration request, pad count
2-3	P_Key	partition key, 0xffff default
5-7	Dest QP	24 bits
8	A	ack request bit
9-11	PSN	24 bits

RETH: 8-byte virtual address, 4-byte rkey, 4-byte DMA length. Present on WRITE FIRST and ONLY, and on READ REQUEST.

RC opcodes decoded by bth_parse.py

opcode	name
0x00 to 0x05	SEND FIRST, MIDDLE, LAST, LAST_WITH_IMM, ONLY, ONLY_WITH_IMM
0x06 to 0x0B	RDMA_WRITE FIRST, MIDDLE, LAST, LAST_WITH_IMM, ONLY, ONLY_WITH_IMM
0x0C	RDMA_READ_REQUEST
0x0D to 0x10	RDMA_READ_RESPONSE FIRST, MIDDLE, LAST, ONLY
0x11	ACKNOWLEDGE
0x12	ATOMIC_ACKNOWLEDGE
0x13, 0x14	COMPARE_SWAP, FETCH_ADD

modify_qp attribute masks, by transition

RESET to INIT: STATE, PKEY_INDEX, PORT, ACCESS_FLAGS.

INIT to RTR: STATE, PATH_MTU, DEST_QPN, RQ_PSN, MAX_DEST_RD_ATOMIC, MIN_RNR_TIMER, AV.

RTR to RTS: STATE, TIMEOUT, RETRY_CNT, RNR_RETRY, SQ_PSN, MAX_QP_RD_ATOMIC.

Common errors and what to ask first

symptom	first question	usual cause
modify_qp EINVAL	which state, which mask	a mask bit missing or extra for that transition
completion status 12 (RETRY_EXC_ERR)	did the peer reach RTR	wrong dest qpn or gid, or fabric loss
completion status 13 (RNR_RETRY_EXC_ERR)	did the peer post RECVs	receiver slower than sender
completion status 10 (REM_ACCESS_ERR)	which access flags on the peer QP and MR	REMOTE_WRITE or rkey missing
port not ACTIVE on rxe	does the netdev have carrier and an address	link down
numbers look too good on rxe	busy polling	the VM cores are shared with the emulation

Repository map

path	role
setup/	VM bring-up, pod preflight, counter baseline
verbs/handshake.py	TCP exchange of the connection record
verbs/connection.py	explicit RC QP state machine
verbs/pingpong.py, write_read.py, atomics.py, multiqp.py	one module per investigation
forensics/counters.py	sysfs counter snapshot and diff
forensics/bth_parse.py	RoCEv2 BTH and RETH decoder
forensics/inject.py	netem loss, MTU, no-recv flag, each with undo
bench/bench.py, plots.py	size sweep to CSV, two-panel SVG
casefiles/	one directory per experiment
tests/	hardware-free pytest suite

Reading list

- rdma-core: `man ibv_modify_qp`, `man ibv_post_send`, `man ibv_reg_mr`, and `libibverbs/examples/rc_pingpong.c`.
- InfiniBand Architecture Specification, volume 1, release 1.4: chapter 9 (transport layer, BTH and RETH definitions), chapter 10 (verbs), and annex A16 (RoCE), A17 (RoCEv2).
- RFC 5040 (RDMA Protocol Specification) for the iWARP contrast.
- NVIDIA "RDMA Aware Networks Programming User Manual" for the mlx5 counter descriptions.
- The DCQCN paper (Zhu et al., SIGCOMM 2015) to connect `np_cnp_sent` and `rp_cnp_handled` to what fabric-forensics measures on the switches.